# ロール管理画面UI設計書

## メタデータ

**title: ロール管理画面UI設計書**
**category: UI設計**
**version: 1.0.0**
**last_updated: 2026-02-02**
**author: Development Team**
**tags: [system-admin, role-management, master-data, ui]**

## 関連ドキュメント

- ロール追加ダイアログ
- ロール編集ダイアログ
- 機能管理ダイアログ
- ロール管理API仕様
- マスタ管理規約
- UIポリシー

---

## 1. 概要

### 1.1. 目的

システム管理者がロールマスタを管理するための画面です。ロールの新規追加、編集、アクティブ/非アクティブ切替、一覧表示、CSV操作などの機能を提供します。

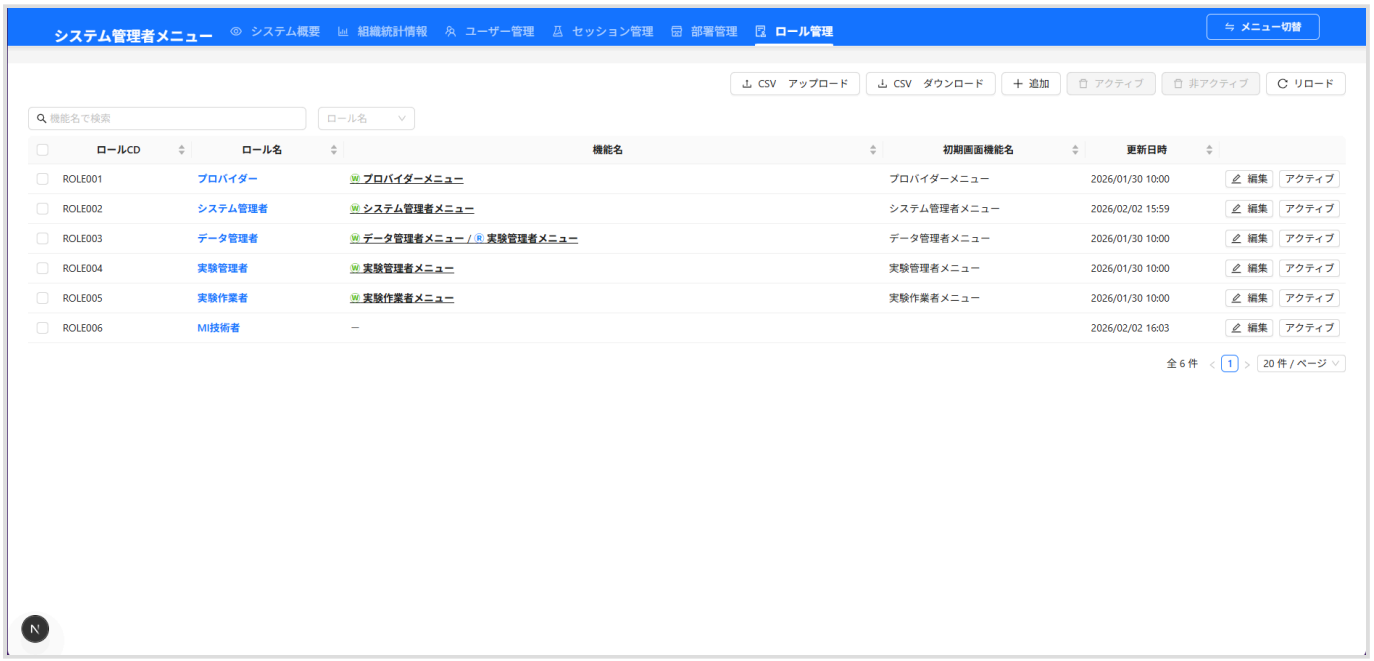
### 1.2. 利用可能な権限

- **System Admin（システム管理者）**: 全機能利用可能

## 1.3. 画面情報

| 項目 | 値 |
|---|---|
| URL | `/system-admin` (システム管理者メニュー) → ロール管理タブ |
| 物理ファイル名 | `RoleManagement.tsx` |
| 配置 | `frontend/src/components/SystemManagement/` |
| 親コンポーネント | システム管理者メニュー |

## 1.4. 画面イメージ

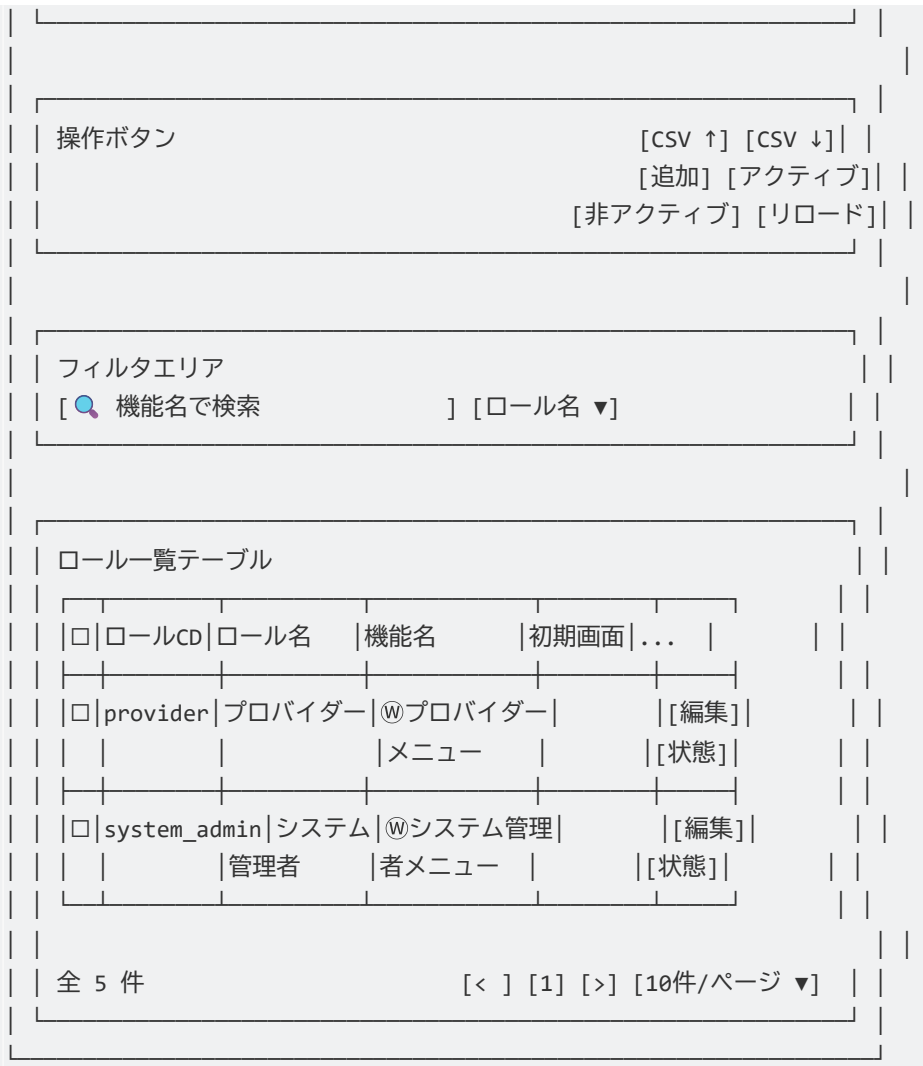



---

# 2. 画面構成

## 2.1. レイアウト（デスクトップ）

```
┌──────────────────────────────────────────────────────────────┐
│ [システム管理者メニュー]                                      │
│ ┌──────────────────────────────────────────────────────────┐ │
│ │ [ロール管理タブ] [部署管理タブ]                          │ │
```




## 2.2. レイアウト（モバイル）



```
│ │ (カード形式)          │ │
│ └──────────────────┘ │
└──────────────────────┘
```

---

# 3. ツールバー

## 3.1. 操作ボタン配置

**配置**: 画面右上

**ボタン一覧**:

| ボタン | アイコン | 機能 |
|---|---|---|
| CSVアップロード | `<UploadOutlined />` | CSV一括登録（将来実装） |
| CSVダウンロード | `<DownloadOutlined />` | CSV一括ダウンロード |
| 追加 | `<PlusOutlined />` | 新規ロール追加 |
| アクティブ | `<DeleteOutlined />` | 選択したロールをアクティブ化 |
| 非アクティブ | `<DeleteOutlined />` | 選択したロールを非アクティブ化 |
| リロード | `<ReloadOutlined />` | データ再読み込み |

## 3.2. ボタン仕様

### CSVアップロード

**仕様**:

- **状態**: 現在は非活性（将来実装予定）
- **動作**: CSVファイルを選択して一括登録

### CSVダウンロード

**コンポーネント**: `<Button>`

```
<Button icon={<DownloadOutlined />} onClick={handleDownload}>
  CSV<br />ダウンロード
</Button>
```

**動作**:

1. 現在のフィルター条件を適用
2. フィルタリング済みデータをCSV形式でエクスポート
3. ファイル名: `ロール管理_YYYYMMDD-HHmmss.csv`
4. UTF-8 BOM付きで出力

## 追加ボタン

**コンポーネント**: `<Button>`

```
<Button icon={<PlusOutlined />} onClick={handleAdd}>
  追加
</Button>
```

**動作**:

1. ロール追加ダイアログを表示
2. 詳細は ui_role_add_dialog.md を参照

## アクティブボタン

**コンポーネント**: `<Button>`

```
<Button
  icon={<DeleteOutlined />}
  onClick={handleActivate}
  disabled={selectedRowKeys.length === 0}
  style={{
    minWidth: 85,
    backgroundColor: selectedRowKeys.length > 0 ? token.colorSuccess : undefined,
    borderColor: selectedRowKeys.length > 0 ? token.colorSuccess : undefined,
    color: selectedRowKeys.length > 0 ? token.colorWhite : undefined,
```

```
  }}
>
  アクティブ
</Button>
```

**動作**:

1. 選択された行のロールを一括でアクティブ化
2. 確認モーダルを表示
3. OK時、API呼び出し（各ロールに対して順次実行）
4. 成功メッセージ表示
5. データ再読み込み

## 非アクティブボタン

**コンポーネント**: `<Button>`

```
<Button
  danger
  icon={<DeleteOutlined />}
  onClick={handleDeactivate}
  disabled={selectedRowKeys.length === 0}
  style={{
    minWidth: 85,
    backgroundColor: selectedRowKeys.length > 0 ? token.colorWarning : undefined,
    borderColor: selectedRowKeys.length > 0 ? token.colorWarning : undefined,
    color: selectedRowKeys.length > 0 ? token.colorWhite : undefined,
  }}
>
  非アクティブ
</Button>
```

**動作**:

1. 選択された行のロールを一括で非アクティブ化
2. 確認モーダルを表示（危険操作として赤色）
3. OK時、API呼び出し
4. 成功メッセージ表示
5. データ再読み込み

### リロードボタン

**コンポーネント**: `<Button>`

```
<Button icon={<ReloadOutlined />} onClick={handleReload}>
  リロード
</Button>
```

**動作**:

1. APIから最新データを再取得
2. フィルター条件をリセット
3. 選択状態をクリア

---

# 4. フィルタリング機能

## 4.1. 配置

**位置**: ツールバーの下、テーブルの上

**レイアウト**:

```
<Row gutter={16} style={{ marginBottom: 8 }}>
  <Col style={{ width: 400 }}>
    {/* 機能名検索 */}
  </Col>
  <Col style={{ width: 150 }}>
    {/* ロール名フィルター */}
  </Col>
</Row>
```

## 4.2. 機能名検索（曖昧検索）

**コンポーネント**: `<Input>`

**仕様**:

| 項目 | 値 |
| --- | --- |
| 幅 | 400px |
| プレースホルダー | 「機能名で検索」 |
| 前置アイコン | `<SearchOutlined />` |
| クリア可能 | `allowClear={true}` |

**検索対象**:

- 機能名フィールドのみ（`functionName`）

**検索方式**:

- 部分一致（大文字小文字を区別しない）
- リアルタイム検索（入力ごとにフィルタリング）

**実装例**:

```
<Input
  placeholder="機能名で検索"
  prefix={<SearchOutlined />}
  value={searchText}
  onChange={(e) => setSearchText(e.target.value)}
  allowClear
  style={{ width: 400 }}
/>
```

## 4.3. ロール名フィルター

**コンポーネント**: `<Select>`

**仕様**:

| 項目 | 値 |
| --- | --- |
| 幅 | 150px |
| プレースホルダー | 「ロール名」 |
| クリア可能 | `allowClear={true}` |

**選択肢**:

- プロバイダー
- システム管理者
- データ管理者
- 実験管理者
- 実験作業者
- MI技術者

**表示順序**: 上記の固定順（五十音順ではない）

**フィルター方式**:

- 完全一致
- 単一選択

**実装例**:

```
<Select
  placeholder="ロール名"
  value={selectedRoleNameFilter || undefined}
  onChange={(value) => setSelectedRoleNameFilter(value || '')}
  allowClear
  style={{ width: 150 }}
  options={[
    { label: 'プロバイダー', value: 'プロバイダー' },
    { label: 'システム管理者', value: 'システム管理者' },
    { label: 'データ管理者', value: 'データ管理者' },
    { label: '実験管理者', value: '実験管理者' },
    { label: '実験作業者', value: '実験作業者' },
    { label: 'MI技術者', value: 'MI技術者' },
  ]}
/>
```

---

# 5. テーブル一覧

## 5.1. テーブル仕様

**コンポーネント**: `<Table>`

**仕様**:

| 項目 | 値 |
|---|---|
| サイズ | `small` |
| 行選択 | チェックボックスあり |
| ページング | 20件/ページ（デフォルト） |
| スクロール | 横スクロール有効（x: 1400） |

## 5.2. カラム定義

| カラム名 | データ項目 | 幅 | 配置 | ソート | 備考 |
|---|---|---|---|---|---|
| （チェックボックス） | - | 50px | center | - | 行選択用 |
| ロールCD | `role` | 150px | left | ○ | 主キー |
| ロール名 | `roleName` | 170px | left | ○ | 強調表示 |
| 機能名 | `functionName` | 600px | left | ○ | クリック可能、権限アイコン付き |
| 初期画面機能名 | `initialScreenFunctionName` | 225px | left | ○ | - |
| 更新日時 | `updatedAt` | 150px | right | ○ | YYYY/MM/DD HH:mm形式 |
| （アクション） | - | 140px | right | - | 編集・状態切替ボタン |

## 5.3. 機能名カラムの特殊表示

**表示形式**:

```
Ⓦシステム管理者メニュー / ⓇSystem Administrator Menu
```

**権限アイコン**:

- Ⓦ: 書込可能（writable）

- Ⓡ: 読取専用（read-only）

**表示ルール**:

1. 機能が設定されていない場合: 「－」（灰色、下線なし）
2. 機能が設定されている場合: 機能名を表示（下線あり、クリック可能）
3. 複数機能の場合: `/`（スペース+スラッシュ+スペース）で区切る

**クリック動作**:

1. 機能名をクリック
2. 機能管理ダイアログを開く（directモード）
3. DBから最新のロール情報を取得して表示

**実装例**:

```
render: (text: unknown, record: MasterRecord) => {
  const functions = (record.functions as Array<{ name: string; icon: string; color: string }>)
|| [];

  return (
    <span
      style={{
        cursor: 'pointer',
        textDecoration: functions.length > 0 ? 'underline' : 'none'
      }}
      onClick={() => handleOpenFunctionModalDirect(record)}
    >
      {functions.length === 0 ? (
        <span style={{ color: token.colorTextSecondary }}>－</span>
      ) : (
        functions.map((func, idx) => (
          <React.Fragment key={idx}>
            {idx > 0 && <span style={{ color: token.colorText }}> / </span>}
            <span style={{ color: func.color, fontWeight: token.fontWeightStrong }}>
{func.icon}</span>
            <span style={{ color: token.colorText, fontWeight: token.fontWeightStrong }}>
{func.name}</span>
          </React.Fragment>
        ))
      )}
    </span>
```

```
  );
}
```

## 5.4. アクションカラム

**配置**: 右端固定（`fixed: 'right'`）

**ボタン構成**:

```
<Flex gap="small" justify="end">
  <Button
    size="small"
    icon={<EditOutlined />}
    onClick={() => handleEdit(record)}
  >
    編集
  </Button>
  <Button
    size="small"
    onClick={() => handleToggleActive(record)}
    danger={record.isActive === false}
    style={{ minWidth: '80px' }}
  >
    {record.isActive !== false ? 'アクティブ' : '非アクティブ'}
  </Button>
</Flex>
```

**編集ボタン**:

- クリック時: ロール編集ダイアログを開く
- 詳細は ui_role_edit_dialog.md を参照

**状態切替ボタン**:

- 現在の状態を表示（アクティブ/非アクティブ）
- クリック時: 確認モーダルを表示後、状態を切り替え
- 非アクティブ時は `danger` スタイル（赤色）

## 5.5. ページング

**仕様**:

```
pagination={{
  pageSize: 20,
  showSizeChanger: true,
  pageSizeOptions: ['10', '20', '50', '100'],
  showTotal: (total) => `全 ${total} 件`,
}}
```

**デフォルト**: 20件/ページ

**選択可能なページサイズ**: 10, 20, 50, 100

---

# 6. API仕様

## 6.1. ロール一覧取得

**エンドポイント**: `GET /api/v1/systemManagement/role`

**リクエストヘッダー**:

```
Authorization: Bearer {session_token}
```

**レスポンス（成功時）**:

```
{
  "success": true,
  "data": [
    {
      "roleId": "provider",
      "roleName": "プロバイダー",
      "functionName": "ⓦプロバイダーメニュー",
      "initialScreenFunctionName": null,
      "functionWritePermissions": "プロバイダーメニュー",
      "functions": [
        {
          "name": "プロバイダーメニュー",
          "icon": "ⓦ",
```

```
          "color": "#52c41a"
        }
      ],
      "functionIds": ["010000"],
      "writableFunctionIds": ["010000"],
      "isActive": true,
      "updatedAt": "2026-02-02T09:00:00.000Z"
    },
    {
      "roleId": "system_admin",
      "roleName": "システム管理者",
      "functionName": "ⓦシステム管理者メニュー",
      "initialScreenFunctionName": null,
      "functionWritePermissions": "システム管理者メニュー",
      "functions": [
        {
          "name": "システム管理者メニュー",
          "icon": "ⓦ",
          "color": "#52c41a"
        }
      ],
      "functionIds": ["020000"],
      "writableFunctionIds": ["020000"],
      "isActive": true,
      "updatedAt": "2026-02-02T09:00:00.000Z"
    }
  ],
  "meta": {
    "timestamp": "2026-02-02T09:00:00.000Z",
    "requestId": "req_abc123"
  }
}
```


## 6.2. ロールステータス更新（個別）

**エンドポイント**: `PATCH /api/v1/systemManagement/role/:roleId/status`

**リクエストボディ**:


```
{
  "isActive": true
}
```


**レスポンス（成功時）**:

```
{
  "success": true,
  "meta": {
    "timestamp": "2026-02-02T09:00:00.000Z",
    "requestId": "req_abc123"
  }
}
```

## 6.3. CSVダウンロード

**エンドポイント**: `GET /api/v1/systemManagement/role/download-csv`

**クエリパラメータ**:

| パラメータ | 型 | 必須 | 説明 |
|---|---|---|---|
| `search` | string | - | 検索文字列 |
| `roleName` | string | - | ロール名フィルター |

**レスポンス**: CSV形式のファイル（`text/csv; charset=utf-8`）

**CSVフォーマット**:

```
ロールCD,ロール名,機能名,初期画面機能名,状態,更新日時
provider,プロバイダー,Ⓦプロバイダーメニュー,,アクティブ,2026/02/02 09:00
system_admin,システム管理者,Ⓦシステム管理者メニュー,,アクティブ,2026/02/02 09:00
```

---

# 7. エラーハンドリング

## 7.1. エラーメッセージ一覧

| エラーコード | メッセージ | 説明 |
|---|---|---|
| `AUTH_UNAUTHORIZED` | 「認証エラーが発生しました」 | 未認証 |
| `AUTH_FORBIDDEN` | 「この操作を実行する権限がありません」 | 権限不足 |

| | | |
|---|---|---|
| `RESOURCE_NOT_FOUND` | 「指定されたロールが見つかりません」 | ロールが存在しない |
| `SYSTEM_ERROR` | 「システムエラーが発生しました」 | サーバーエラー |

## 7.2. エラー表示

**コンポーネント**: `message.error()` (Ant Design App)

**表示位置**: 画面上部中央

**表示時間**: 3秒

**実装例**:

```
try {
  await fetchRoleMasters();
} catch (error) {
  console.error('API fetch error:', error);
  message.error(`ロールマスタの取得に失敗しました: ${error instanceof Error ? error.message :
'Unknown error'}`);
  setMasters([]);
}
```

---

# 8. レスポンシブデザイン

## 8.1. ブレークポイント

| サイズ | 幅 | テーブル表示 | フィルター配置 |
|---|---|---|---|
| xs | < 576px | カード形式 | 縦積み |
| sm | 576px ~ 768px | カード形式 | 縦積み |
| md | 768px ~ 992px | テーブル | 横並び |
| lg | 992px ~ 1200px | テーブル | 横並び |
| xl | 1200px ~ 1600px | テーブル | 横並び |
| xxl | > 1600px | テーブル | 横並び |

## 8.2. モバイル対応

**テーブル → カード変換**:

- 576px未満でカード表示に切り替え
- 各カードにロール情報を表示
- アクションボタンはカード下部に配置

**ツールボタンの配置**:

- 縦積み配置
- ボタン幅: 100%
- 間隔: 8px

---

# 9. アクセシビリティ

## 9.1. キーボード操作

**Tab順序**:

1. フィルター入力欄
2. ツールボタン
3. テーブル行（チェックボックス、編集ボタン、状態ボタン）
4. ページネーション

**ショートカットキー**:

- Enter: 編集ボタンのクリック（フォーカス時）
- Space: チェックボックスの切り替え

## 9.2. スクリーンリーダー対応

**ARIA属性**:

```
<Table
  aria-label="ロール一覧"
  rowKey="role"
  {...otherProps}
/>
```

**アクションボタン**:

```
<Button
  aria-label={`${record.roleName}を編集`}
  onClick={() => handleEdit(record)}
>
  編集
</Button>
```

## 9.3. コントラスト比

| 要素 | 背景色 | 前景色 | コントラスト比 |
|---|---|---|---|
| 通常テキスト | `#FFFFFF` | `#000000` | 21:1 ✅ |
| 機能名（強調） | `#FFFFFF` | `#CF0027` | 4.8:1 ✅ |
| 権限アイコン（Ⓦ） | `#FFFFFF` | `#52c41a` | 4.5:1 ✅ |
| ボタン（Primary） | `#CF0027` | `#FFFFFF` | 4.8:1 ✅ |

**WCAG 2.1 AA基準（4.5:1以上）を満たす**

---

# 10. パフォーマンス最適化

## 10.1. データ取得の最適化

**初回ロード時**:

1. ロール一覧を取得（`fetchRoleMasters()`）
2. ローディング状態を表示

3. データを状態管理に格納

**フィルタリング**:

- クライアントサイドフィルタリング（`useMemo`）
- APIへの再リクエストなし

## 10.2. メモ化

**フィルタリング結果**:

```
const filteredMasters = useMemo(() => {
  return masters.filter(master => {
    const matchesSearch = !searchText.trim() ||
      String(master.functionName).toLowerCase().includes(searchText.toLowerCase());

    const matchesRoleNameFilter = !selectedRoleNameFilter ||
      String(master.roleName) === selectedRoleNameFilter;

    return matchesSearch && matchesRoleNameFilter;
  });
}, [masters, searchText, selectedRoleNameFilter]);
```

**カラム定義**:

```
const columns = useMemo(() => {
  // カラム定義を動的に生成
}, [selectedMasterInfo, token]);
```

---

# 11. 実装例（TypeScript + Next.js + Ant Design）

## 11.1. メインコンポーネント

```
'use client';
```

```tsx
import React, { useState, useMemo, useEffect } from 'react';
import {
  Table,
  Button,
  Input,
  Flex,
  Select,
  App,
  Typography,
  Row,
  Col,
  theme,
} from 'antd';
import {
  SearchOutlined,
  PlusOutlined,
  DeleteOutlined,
  ReloadOutlined,
  UploadOutlined,
  DownloadOutlined,
  EditOutlined,
} from '@ant-design/icons';
import type { ColumnsType } from 'antd/es/table';
import { formatDateTime } from '@/utils/dateFormatter';
import { getRoleConfig } from '@/types/roleTypes';
import {
  fetchMasters,
  updateMasterStatus,
  batchUpdateMasterStatus,
  downloadCsv,
  type MasterRecord,
} from '@/services/roleApiService';

const { Text } = Typography;

export default function RoleManagement() {
  const { token } = theme.useToken();
  const { message, modal } = App.useApp();

  // State管理
  const [masters, setMasters] = useState<MasterRecord[]>([]);
  const [loading, setLoading] = useState<boolean>(false);
  const [searchText, setSearchText] = useState('');
  const [selectedRoleNameFilter, setSelectedRoleNameFilter] = useState
```